

Technologies for IoT Ecosystems

Software Lab

INTRODUCTION

This laboratory course guides you through the design and implementation of a complete Smart Home IoT ecosystem built entirely in Python and, in the final part, extended to physical hardware (i.e. the Arduino). Across thirteen progressive exercises (split in three parts), you will construct every layer of a real-world IoT platform: from the REST endpoints that expose individual sensor readings (Software Lab – Part 1), through the centralized Catalog that ties the platform together (Software Lab – Part 2), up to a fully distributed controller that reacts autonomously to physical sensor data and drives actuators on an Arduino board (Software Lab – Part 3).

This is not a collection of isolated exercises. Each component you build is a functional piece of a larger system. This explicitly requires you to run multiple services simultaneously and to distribute the IoT ecosystem across several computer systems. By the final session all thirteen components will cooperate over both REST and MQTT, with a physical Arduino as a first-class citizen of the ecosystem.

LEARNING OBJECTIVES

By the end of this lab you will be able to:

- Apply Object-Oriented Programming principles throughout.
- Build a centralized IoT Catalog that acts simultaneously as a Service Registry and a Device Registry, persisting its state to a JSON file, automatically purging stale registrations via a background thread, and exposing its interface over both REST and MQTT.
- Implement bidirectional MQTT communication using the paho-mqtt library, including topic-based pub/sub, QoS levels, dynamic configuration, and device discovery from the Catalog.

TECHNICAL PREREQUISITES

Before starting, ensure the following are installed in your Python environment:

```
pip install cherrypy requests paho-mqtt
```

The MQTT broker to use is the public Eclipse Foundation broker:

```
host: iot.eclipse.org  
port: 1883
```

You do not need to run a local broker. However, because this is a public broker, prefix all your team's topics with a unique group identifier (e.g. /tiot/groupXX/...) to avoid collisions with other teams. All services log to stdout; open a separate terminal window for each service to monitor the system in real time.

Exercise 05 — IoT Catalog — REST Service Registry

Develop an IoT Catalog that acts as a Service Registry and Device Registry for the smart home ecosystem. All actors (devices, services) must register themselves and periodically refresh their registration. The Catalog persists all data to a JSON file on disk and exposes a complete REST API. For each action in the Catalog Choose the right HTTP Method among (GET, POST, PUT and DELETE). A background thread automatically removes stale registrations (last_update older than 2 minutes).

Possible BONUS: replace the JSON to Catalog information with a database. Data exchange among the different actors must be guaranteed with REST API

Requirements

1. Define a Catalog class (CherryPy) with full CRUD for devices and services .
2. Persist all data to catalog.json; reload on startup.
Possible BONUS: replace the JSON with a database
3. Each device/service record must include:
 - Unique ID of the device/service,
 - Description of it,
 - REST endpoint URL exposed by the device/service (if present),
 - MQTT info (ip, port, topic) used by the device/service (if present),
 - Resources list,
 - Insert_timestamp (managed by Catalog only).
4. Register and then update a new device/service.
The system must check for existing ID: if found, update insert_timestamp (refresh); if not, create new record.
5. Launch a background thread (threading.Thread) that every 60 seconds removes entries for both device/service where insert_timestamp is older than 120 seconds.
6. Retrieve, via a specific Catalog endpoint, the MQTT broker connection info stored in catalog.json.

Tips & Notes

- ⚠ Use `threading.Thread(target=self._cleanup_loop, daemon=True).start()` in `__init__`.
- ⚠ Protect `catalog.json` read/write with `threading.Lock()` to avoid race conditions.
- ⚠ Store broker info in a top-level 'broker' key in `catalog.json`:
`{"ip":"localhost","port":1883}`.

Test all endpoints with curl or Postman.

Exercise 06 — IoT Catalog — REST Client & Service Auto-Registration

Develop a Python REST client that interacts with the Catalog built in Exercise 05. The client must demonstrate all Catalog REST APIs. Additionally, extend the smart home services from Exercises 01–04 (see Software Lab Part 1) so that each service auto-registers itself with the Catalog on startup and periodically refreshes its registration every 60 seconds using a background thread.

Requirements	1. Define a <code>CatalogClient</code> class that wraps the requests library for all Catalog REST calls. 2. Implement methods: <code>get_catalog()</code>, <code>get_devices()</code>, <code>get_device(id)</code>, <code>get_broker()</code>, <code>register_device(payload)</code>, <code>refresh_device(id)</code>, <code>register_service(payload)</code>, <code>refresh_service(id)</code>. 3. Extend <code>SmartHomeSensorService</code> (Ex. 01/02) and <code>ActuatorService</code> (Ex. 03) to call <code>CatalogClient.register_service()</code> on startup (see Software Lab Part 1). 4. Add a background thread in each service that calls <code>CatalogClient.refresh_service()</code> every 60 seconds. 5. Handle connection errors gracefully: log a warning and retry on the next cycle.
--------------	--

Exercise 07 — IoT Catalog — MQTT Extension

Extend the Catalog from Exercise 05 to also accept device and service registrations over MQTT, in addition to REST. The Catalog subscribes to a well-known registration topic (defined by your team), processes incoming registration records, and publishes a acknowledgement to a device-specific or service-specific response topic. This exercise introduces the dual REST+MQTT interface pattern common in production IoT platforms.

Requirements	Add an <code>MQTTCatalogBridge</code> class that holds a <code>paho-mqtt</code> Client instance that will start alongside the CherryPy server in a second thread.
Tips & Notes	⚠ Use the message broker provided by eclipse foundation: • host: <code>iot.eclipse.org</code> • port: <code>1883</code> Test all the MQTT endpoints (both as publisher and subscriber) with Mosquitto clients (to install in your PC)

Exercise 08 — IoT Catalog — MQTT Device Client

Develop a Python MQTT client that emulates an IoT device interacting with the Catalog's MQTT interface from Exercise 07. The client must periodically register or refresh its registration, subscribe to the Catalog's ACK topic to receive confirmation, and support interactive queries (get all devices, get specific device) over MQTT

Requirements	1. Define a <code>DeviceMQTTClient</code> class with <code>paho-mqtt</code>; start with a unique <code>client_id</code>. 2. On connect, subscribe to the personal ACK topic (defined by your team) and to the query response topic. 3. Launch a background thread that every 60 seconds builds and publishes a registration record. 4. Provide an interactive menu: (1) register now, (2) query all devices, (3) query device by ID, (4) quit.
Tips & Notes	⚠ Test by running Exercises 07 and 08 simultaneously ⚠ Use the message broker provided by eclipse foundation: • host: <code>iot.eclipse.org</code> • port: <code>1883</code>

Test all the MQTT endpoints (both as publisher and subscriber) with Mosquitto clients (to install in your PC)

Exercise 09 — MQTT Temperature Publisher

Develop an MQTT publisher that emulates a physical IoT temperature sensor node. On startup, the node queries the Catalog (Exercise 05) via REST to retrieve the MQTT broker address, then registers itself. Every 30 seconds it publishes a SenML temperature record to its registered MQTT topic. The node also subscribes to a command topic to receive dynamic configuration (e.g. change of publish interval).

This MQTT Temperature Publisher must register itself on the Catalog via REST and keep the registration periodically updated (see Exercise 05)

Tips & Notes

⚠ Use the message broker provided by eclipse foundation:

- host: iot.eclipse.org
- port: 1883

Test all the MQTT endpoints (both as publisher and subscriber) with Mosquitto clients (to install in your PC)